Artificial Morphogenesis using Evolutionary Cellular Automata: A Study of Fitness Function Design

William Greenwood
University of Tennessee
Knoxville, USA
wgreenwo@vols.utk.edu

Phil Marino
University of Tennessee
Knoxville, USA
pmarino@vols.utk.edu

Ethan Weathers
University of Tennessee
Knoxville, USA
dweathe5@vols.utk.edu

***Abstract*—Artificial morphogenesis studies how complex spatial structures can emerge from simple local interactions. While recent approaches have used neural cellular automata and gradient-based optimization to generate such patterns, these methods are often difficult to interpret and analyze.**

In this work, we investigate whether artificial morphogenesis can instead be achieved using evolutionary algorithms applied to binary cellular automata rule sets. Each candidate solution encodes a complete CA transition rule, and evolution is guided by a target-matching fitness function. We evaluate three fitness formulations: pixel-level similarity using Intersection over Union (IoU), boundary-based similarity using Boundary IoU, and global structural similarity using distance transforms.

We conduct multiple evolutionary runs across seven target patterns using a fixed-grid CA system initialized from a single seed cell. Our results show that fitness function design significantly influences convergence behavior in evolutionary cellular automata. We also observe that high numerical fitness does not always correspond to visually meaningful pattern formation, indicating instances of deceptive convergence. Finally, we identify that optimal CA steps were consistently between 7–25 iterations.

Overall, our findings suggest that fitness design is a critical factor in evolutionary morphogenesis, and that naïve similarity metrics may lead to structurally incorrect yet highly fit solutions.

Code, experiments, and visualizations are publicly available in our project repository.

***Index Terms*—artificial morphogenesis, cellular automata, evolutionary algorithms, fitness function design**

I. Introduction and Motivation

Artificial morphogenesis studies how complex structures can emerge from simple local interactions. Recent work has demonstrated that neural cellular automata can grow structured patterns from a single seed cell, highlighting the potential for learned morphogenetic systems [1]. However, these approaches often rely on deep neural networks and gradient-based optimization, resulting in systems that are difficult to interpret.

In this work, we investigate whether similar morphogenetic behavior can be achieved using simpler, fully discrete systems. Specifically, we explore the evolution of binary cellular automata (CA) to generate target patterns from a single seed cell on a fixed 17×17 grid.

Cellular automata are discrete mathematical systems in which each cell updates its state based on the configuration of a local neighborhood. Despite their simple formulation, cellular automata are known to exhibit rich behavior, including pattern formation. Foundational work has shown that even simple rule sets can produce highly complex global dynamics, often operating near the "edge of chaos," where structured behavior emerges from local interactions [2], [3].

However, designing effective CA rule sets manually is infeasible due to the size of the search space. For a binary CA with a Moore neighborhood, each rule set consists of $2^9 = 512$ possible neighborhood configurations. This combinatorial complexity motivates the use of evolutionary algorithms (EA) to automatically discover effective rule sets [4].

Evolutionary algorithms optimize candidate solutions through iterative processes of selection, mutation, and crossover. In this study, each candidate solution corresponds to a complete CA rule set, and evolution is guided by a fitness function that measures similarity to a target pattern.

A central challenge in this setting is fitness function design. Evolutionary algorithms will optimize whatever objective is provided, even if that objective does not align with the desired visual outcome. As a result, poorly designed fitness functions may produce high numerical scores while failing to generate visually-correct patterns.

To address this challenge, we evaluate three fitness functions based on different similarity metrics: pixel-level overlap using Intersection over Union (IoU) [5], boundary-based similarity using Boundary IoU [6], and global structural similarity using distance transforms [7]–[9].

We selected this problem to better understand how more interpretable systems can replicate morphogenetic behavior typically achieved by more complex neural approaches. To investigate artificial morphogenesis using evolutionary cellular automata, this study addresses the following research questions:

- **RQ1 (Fitness Function Design):** How does fitness function design influence convergence behavior in evolutionary cellular automata?

- **RQ2 (Visual Solution Quality):** Do higher fitness scores correspond to visually meaningful morphogenetic patterns?
- **RQ3 (CA Runtime Dynamics):** Is there an optimal number of cellular automata iterations for stable pattern formation?

II. Related Work

A. Artificial Morphogenesis

Artificial morphogenesis studies how complex structures can emerge from simple local interactions. A major recent example is the work of Mordvintsev et al. [1], who showed that neural cellular automata can grow target shapes from a single seed cell. Their work is closely related to ours because it demonstrates morphogenetic growth in a grid-based system. However, their approach relies on gradient-based optimization of neural update rules, whereas our work evolves discrete cellular automata rule sets using evolutionary algorithms.

More recent work has further expanded morphogenetic systems toward large-scale and open-ended evolution in continuous cellular automata [10]. These approaches demonstrate the growing complexity and capability of modern morphogenetic models, but often rely on significantly more complex representations. In contrast, our work focuses on understanding how simple discrete systems behave under different fitness formulations.

B. Evolutionary Algorithms

Evolutionary algorithms optimize candidate solutions through selection, crossover, and mutation. They are particularly useful when the search space is too large for manual design. Mitchell et al. [4] reviewed how genetic algorithms have been used to evolve cellular automata for global computational behavior. Our work builds on this idea, but applies evolutionary search specifically to morphogenesis toward target binary patterns.

C. Cellular Automata

Cellular automata are systems in which each cell updates according to local neighborhood rules, often producing complex global behavior from simple local interactions. Foundational work by Wolfram [2] and Langton [3] established cellular automata as systems capable of rich emergent behavior, particularly near the so-called "edge of chaos."

More recent work has continued to explore CA in artificial life and self-reproducing systems, as demonstrated by Sayama [11]. Cellular automata have also been applied in practical domains such as medical imaging [12]. These works motivate CA as a flexible spatial framework, while our work focuses specifically on evolving CA rules for morphogenetic growth from a single seed.

D. Fitness Design in Evolutionary Systems

Fitness design is critical in evolutionary systems because the algorithm will optimize whatever objective is provided, even if it does not align with the desired visual result. Despite extensive work in evolutionary computation and cellular automata, the role of fitness function design in morphogenetic pattern formation remains relatively underexplored.

To evaluate evolved patterns, we draw on similarity metrics from image analysis. Pixel-level Intersection over Union (IoU) is widely used in segmentation benchmarks [5], while Boundary IoU was introduced to better capture boundary fidelity [6]. Distance transforms have long been used for structural shape comparison [7], [8], and multiscale analysis can improve robustness to spatial variation [9]. These works motivate the three fitness formulations compared in this study.

III. Methodology

To conduct this study, we require core engines for both cellular automata (CA) simulation and evolutionary algorithms (EA). Acting as the genomes in our EA are individual instances of CA rule sets. That is, each genome consists of its own set of CA transition rules, providing the primary mode of exploration within the search space.

A. System Overview

Figure 1 provides a high-level overview of the artificial morphogenesis pipeline. The process begins by initializing a random population of a specified size, forming the first generation for the evolutionary algorithm. Each individual in the population represents a complete CA rule set that governs how patterns evolve over time.

Once initialized, the population is passed into the core evolutionary algorithm. For each generation, every individual is simulated using its corresponding CA rule set for a fixed number of steps. At each step of the simulation, a fitness value is computed according to a specified fitness function. This enables the selection of a final representative fitness for each individual based on its best-performing state during evolution. This approach mitigates issues arising from under- or over-evolved CA behavior.

Following evaluation, a new generation is produced using standard evolutionary operators, including selection, mutation, crossover, and elitism. Tournament selection is used for parent selection, while mutation and crossover rates are specified as configurable parameters. Additionally, elitism is enforced by carrying forward the top-performing individuals from generation n to generation $n + 1$.

This process repeats for a fixed number of generations. Upon completion, experimental data is recorded, including fitness trajectories and final rule sets. These results are stored for further analysis and visualization.

B. Cellular Automata Representation and Implementation

The cellular automata (CA) engine serves as the primary mechanism for image evolution. In this framework, an image is represented as a two-dimensional grid of cells. Each cell is binary-valued, where 0 represents an inactive (white) cell and 1 represents an active (black) cell.

To model artificial morphogenesis, we enforce growth from a single seed cell. Each simulation begins with an initially

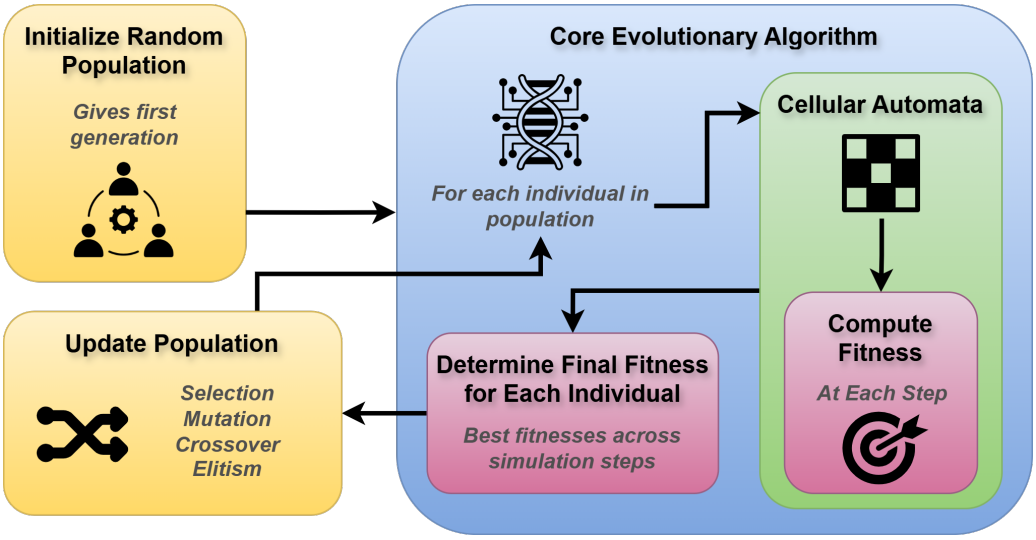


Fig. 1: Overview of the artificial morphogenesis pipeline.

inactive grid, with only the central cell activated. For an $n \times m$ grid with odd dimensions, the initialization is defined as:

$$init_grid = \begin{pmatrix} 0 & \cdots & 0 \\ \vdots & 1 & \vdots \\ 0 & \cdots & 0 \end{pmatrix}$$

Each CA instance evolves according to a fixed rule set. This study focuses exclusively on binary cellular automata using the Moore neighborhood, consisting of a 3×3 region surrounding each cell. Boundary conditions are handled using periodic wrapping. When evaluating neighborhoods at the edges of the grid, cells that fall outside the boundary wrap around to the opposite side of the grid. This effectively creates a toroidal topology, ensuring that all cells maintain a consistent neighborhood size throughout evolution. Since each cell in the neighborhood is binary, this results in $2^9 = 512$ possible neighborhood configurations.

To efficiently encode transition rules, each neighborhood configuration is interpreted as a 9-bit binary number, which is then converted into an integer index. This index is used to query a lookup table of size 512 that defines the next state of the cell. This approach allows each CA rule set to be represented as a fixed-length binary vector.

Finally, to preserve the notion of morphogenetic growth, we enforce an additional constraint: a cell may only activate if at least one neighboring cell is already active. This prevents spontaneous activation far from the evolving structure and ensures growth originates from the seed cell.

C. Target Patterns

To guide evolution through fitness function design, we must establish a diverse set of target patterns. For this study, we selected simple 17×17 binary grids. This size balances computational feasibility with sufficient spatial resolution to allow meaningful morphogenetic growth.

Figure 2 shows the full set of target patterns used in this study. Early experiments initially included geometric shapes such as squares, circles, and hearts of varying sizes. However, these patterns produced nearly identical and consistently poor evolutionary results. This observation led to the realization that effective evaluation requires structurally diverse targets.

In the initial design, geometric shapes differed primarily along their boundaries, resulting in minimal structural variation. To address this limitation, the final pattern set was constructed to span a broader range of structural properties, including differences in density, connectivity, symmetry, and spatial frequency.

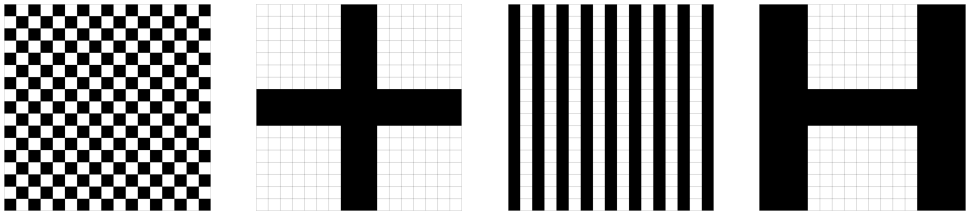
To quantitatively validate this diversity, we compute pairwise similarity between target patterns using a binary Intersection-over-Union (IoU) metric. Lower similarity values indicate greater structural diversity between patterns. The resulting similarity matrix is shown in Figure 3. A detailed description of the IoU metric is provided in Section III-D.

D. Fitness Functions

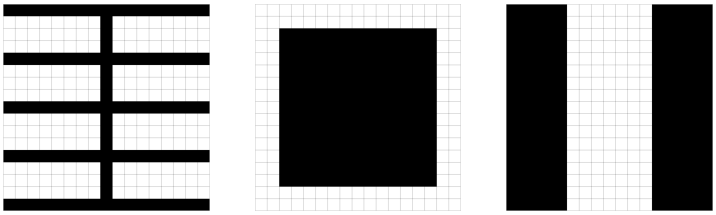
At the core of our study is an evaluation of fitness function design in the evolutionary cellular automata ecosystem. Evolutionary algorithms require a notion of fitness so that candidate solutions can be compared and improved over time. Without a fitness function, there is no mechanism for selection or competition.

However, evolutionary algorithms will optimize whatever objective is defined. This creates a delicate challenge: designing a fitness metric that accurately reflects the desired outcome. A poorly designed fitness function in this study may still achieve near 100% accuracy while producing outputs that bear little visual resemblance to the target patterns.

To address this, we evaluate three similarity metrics of increasing structural sophistication: Pixel IoU, Boundary IoU, and Distance Transform similarity.



(a) Checker (b) Cross (c) Fine Stripes (d) H



(e) Spine (f) Square (g) Stripes

Fig. 2: Target patterns used in the experiments.

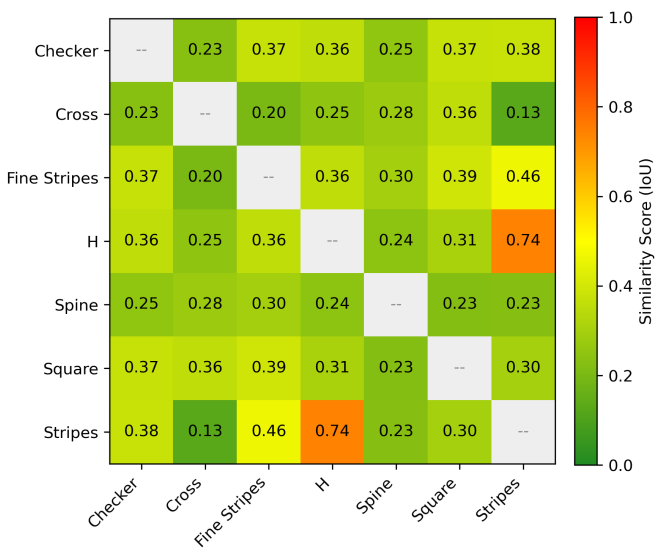


Fig. 3: Binary IoU Similarity Matrix for the target patterns. Lower values indicate higher structural diversity.

1) Pixel IoU: Intersection over Union (IoU), also known as the Jaccard Index, is a commonly used metric for measuring similarity between sets. In image processing and segmentation tasks, IoU measures the overlap between predicted and target pixel regions [5]. Given by (1), IoU computes the ratio between the intersection and union of two binary images:

$$\text{IoU}(A, B) = \frac{A \cap B}{A \cup B} \tag{1}$$

This metric provides a simple yet effective baseline for evaluating pixel-level similarity. Compared to raw pixel accuracy, IoU is less sensitive to small misalignments and better captures overall structural similarity. As a result, IoU has become a standard evaluation metric in image segmentation benchmarks such as the Pascal Visual Object Classes (VOC) challenge [5].

2) Boundary IoU: While pixel IoU captures overall overlap, it may fail to penalize poor boundary alignment. Large regions with incorrect shapes can still achieve high IoU scores, despite lacking visual similarity. To address this limitation, we incorporate Boundary IoU, a metric introduced by Cheng et al. [6], which evaluates similarity specifically along object boundaries.

Boundary IoU computes Intersection over Union over boundary regions rather than entire pixel areas. This shifts emphasis from overall overlap to shape accuracy, encouraging solutions that better match structural features.

Inspired by this approach, we implement a simplified boundary-based metric by extracting edge masks using local neighborhood differences. IoU is then computed between the resulting boundary masks.

In addition to boundary similarity, we incorporate an area matching term to prevent degenerate solutions consisting of thin edge-only structures. The final fitness function combines boundary similarity and area similarity using weighted coefficients:

$$F = 0.6 \cdot \text{BoundaryIoU} + 0.4 \cdot \text{AreaMatch} \tag{2}$$

This hybrid approach balances structural accuracy with overall shape fidelity, encouraging visually meaningful solutions while avoiding trivial edge-based matches.

3) Distance Transform Similarity: To capture global structural similarity, we use a distance-transform-based fitness function. For both the generated and target images, we compute a distance transform using the Euclidean distance to the nearest foreground boundary. The difference between these distance maps provides a measure of morphological dissimilarity that is less sensitive to small pixel misalignments than IoU-based metrics.

To improve robustness to scale variations, we evaluate this measure across multiple downsampled versions of the images (1x, 2x, and 4x resolution where applicable). The final fitness

is computed as the average score across these scales, encouraging consistency in both global shape and coarse structural features.

In summary, we use three fitness metrics with increasingly sophisticated notions of similarity:

- **Pixel IoU:** Raw pixel overlap
- **Boundary IoU:** Structural edges
- **Distance Transform:** Global structure

E. Evolutionary Algorithm Setup

As shown in Fig. 1, the evolutionary algorithm begins with an initialized population of cellular automata genomes. Prior to execution, the program parses a `config.yaml` file containing experiment parameters. The core parameters used in this study are listed below:

- Number of Experiments: 5
- Max Steps: 40
- Generations: 200
- Population Size (N): 80
- Mutation Rate: 2%
- Crossover Rate: 80%
- Tournament Size: 5

The *Max Steps* parameter defines the maximum number of cellular automata iterations allowed when evaluating a genome. The highest fitness observed across all steps is used as the genome's final fitness score.

Additional configuration parameters include the fitness function selection, output directory, random seed, and pattern subset selection. These options enabled flexible experimentation and reproducibility.

Due to the stochastic nature of evolutionary algorithms, multiple experiments were conducted for each configuration. This allowed performance to be evaluated across independent runs and reduced the influence of random initialization.

F. Data Collection

Data collection was performed on a per-experiment basis. For each generation, key statistics were recorded and stored in CSV format. These statistics include:

- `best_in_gen`: Best-performing genome fitness within the generation
- `avg_in_gen`: Average population fitness for the generation
- `best_overall`: Best fitness observed across all generations
- `gen_fitnesses`: List of fitness values for all genomes in the generation
- `gen_steps`: Step at which each genome achieved its highest fitness

Upon completion of each evolutionary run, the best-performing genome was saved as a JSON file containing the learned cellular automata rule set. These saved solutions were then used to generate animated GIFs for qualitative visual analysis of emergent pattern formation. These metrics were later used to analyze convergence behavior and compare fitness function performance.

IV. Results

Our analysis consists of two complementary phases: quantitative and qualitative evaluation. In the quantitative study, we analyze the raw experimental data to evaluate fitness convergence behavior and optimal cellular automata step performance across pattern–fitness combinations. Following this, we perform qualitative analysis by visually comparing the best-evolved solutions against their corresponding targets (Fig. 2).

This dual approach allows us to evaluate not only raw numerical performance, but also whether high fitness scores correspond to visually meaningful results. This distinction highlights a central challenge in fitness function design: ensuring that optimization accurately reflect desired outcomes.

A. Quantitative Results

1) Fitness Convergence: Figure 4a displays fitness convergence across generations for each pattern–fitness combination. These plots illustrate how the best fitness evolves during the evolutionary process.

Across patterns, Pixel IoU exhibits the largest variance between experiments. Many patterns show gradual improvement followed by plateauing around generation 50. In contrast, both Boundary IoU and Distance Transform similarity demonstrate rapid early convergence. However, Boundary IoU often stagnates near its initialization value, while Distance Transform achieves near-optimal fitness almost immediately.

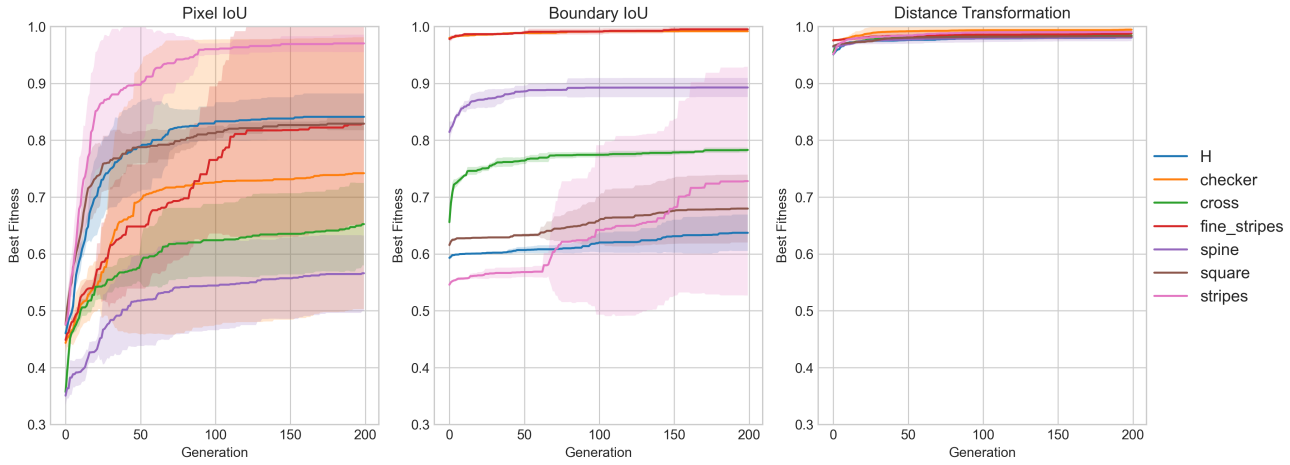
Despite these differences in convergence behavior, Pixel IoU and Boundary IoU produce comparable final fitness ranges (approximately 0.5–1.0). However, the patterns favored by each metric differ. Boundary IoU performs best on patterns with complex boundaries, such as Fine Stripes, Spine, and Cross. Pixel IoU performs better on simpler, block-like patterns such as Stripes, H, and Square.

2) Cellular Automata Step Convergence: Figure 4b presents the convergence of optimal cellular automata steps. This analysis was motivated by an observed instability in evolved solutions when cellular automata were executed for extended durations. Early experiments ran cellular automata for 40–100 steps, however, resulting GIFs often became chaotic, producing misleading fitness evaluations.

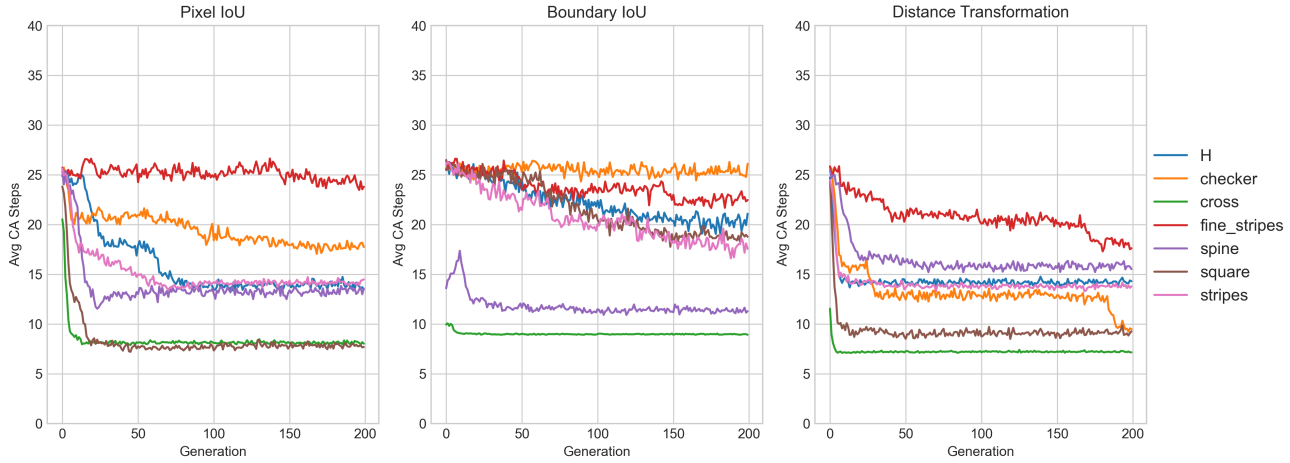
To address this, fitness was evaluated at each cellular automata step. The step achieving the highest fitness was selected as the representative performance for each genome. The optimal step values were then averaged across experiments and plotted over generations.

Results show that, independent of fitness metric, most patterns converge toward a consistent range of optimal steps. Across all configurations, this "sweet spot" occurs between approximately 7 and 25 iterations. These findings suggest that shorter cellular automata runtimes better capture stable pattern formation, while longer runtimes introduce instability and reduce fitness reliability.

These quantitative findings provide insight into convergence behavior and runtime dynamics. However, high fitness scores do not necessarily correspond to visually meaningful solutions.



(a) Best fitness over generations across all pattern–fitness combinations.



(b) Average CA step achieving best fitness over generations.

Fig. 4: Quantitative evaluation of evolutionary performance and CA step dynamics.

To further evaluate solution quality, we next perform qualitative analysis of the best evolved patterns.

B. Qualitative Analysis

To bridge quantitative performance with visually meaningful pattern formation, we conducted a qualitative analysis of the best-evolved solutions. In addition to recording generational statistics in CSV format, each experiment also produced a JSON file containing the evolved cellular automata rule set and its associated fitness value.

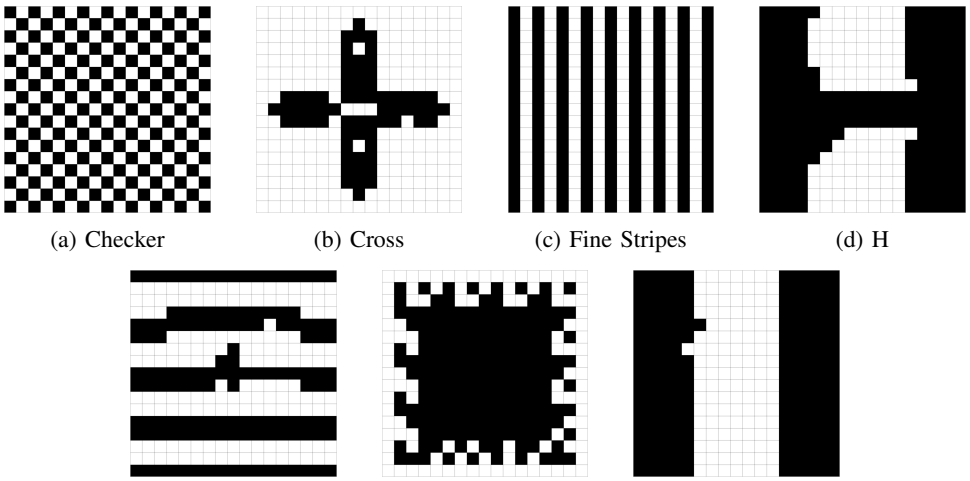
For qualitative evaluation, we selected the experiment achieving the highest fitness for each pattern–fitness combination. This resulted in one candidate solution per combination, yielding a total of 21 evolved patterns for evaluation.

Using the recorded fitness value, we located the corresponding generation within the CSV data. From this generation, we extracted the step at which the highest fitness was achieved. This provided all necessary information to reproduce the evolved pattern: the cellular automata rule set and the optimal number of iterations.

Each selected cellular automaton was then executed for its corresponding number of steps to generate the final evolved pattern. These final frames were visually compared against their target patterns.

For each target pattern, the authors compared the results across the three fitness functions and selected the visually closest match. For example, the evolved checkerboard patterns produced by Pixel IoU, Boundary IoU, and Distance Transform similarity were compared, and the best-performing visual match was selected.

The resulting qualitative winners for each pattern are summarized in Fig. 5 and Table I. We also observed two instances of tied qualitative performance. For the Checker and Fine Stripes patterns, both Distance Transform and Pixel IoU produced identical final frames with perfect fitness scores of 1.0000.



(a) Checker (b) Cross (c) Fine Stripes (d) H

(e) Spine (f) Square (g) Stripes

Fig. 5: Best-evolved patterns per target, selected across all fitness functions based on visual quality.

TABLE I: Fitness, metric, and CA step for the best visually selected pattern per target.

Pattern	**Checker**	**Cross**	**Fine Stripes**	**H**	**Spine**	**Square**	**Stripes**
Fitness Metric	DT (tied Pixel IoU)	DT	DT (tied Pixel IoU)	Pixel IoU	Pixel IoU	Boundary IoU	Pixel IoU
Best Fitness	1.0000	0.9852	1.0000	0.8859	0.6357	0.7609	0.9883
Best Step	9	7	25	13	21	7	13

For the Checker pattern, both methods converged in 9 steps. For Fine Stripes, both achieved perfect fitness, however Distance Transform converged earlier at 25 steps, compared to 39 steps for Pixel IoU.

V. Discussion

In this section, we review our results in detail by answering the following research questions (RQs):

- **RQ1 (Fitness Function Design):** How does fitness function design influence convergence behavior in evolutionary cellular automata?
- **RQ2 (Visual Solution Quality):** Do higher fitness scores correspond to visually meaningful morphogenetic patterns?
- **RQ3 (CA Runtime Dynamics):** Is there an optimal number of cellular automata iterations for stable pattern formation?

A. RQ1: Fitness Function Design

There is a clear relationship between fitness function design and convergence behavior. Quantitatively, Distance Transform similarity dominates across patterns, with most runs converging to near-optimal fitness within the first few generations.

This rapid convergence likely occurs because distance transform similarity measures global structural alignment rather than strict pixel-level accuracy. As a result, many candidate solutions achieve high similarity early in evolution, even if fine structural details are not yet fully developed.

In contrast, Pixel IoU exhibits significantly higher variance across experiments. This behavior is expected because Pixel IoU evaluates exact pixel overlap, making the search landscape more sensitive to small structural differences. Consequently, evolutionary search proceeds more gradually, resulting in slower but often more visually meaningful convergence.

Boundary IoU demonstrates fast initial convergence but frequently stagnates. This behavior suggests that boundary-based metrics may introduce local optima, where partial edge alignment yields moderate fitness but does not strongly encourage further structural refinement.

B. RQ2: Visual Outcomes in Evolutionary CA

While quantitative metrics provide insight into convergence behavior, qualitative analysis reveals how well evolved solutions match the intended morphogenetic patterns. Across patterns, Pixel IoU and Distance Transform most consistently produced visually meaningful results, while Boundary IoU demonstrated strengths in specific structural scenarios.

Pixel IoU performed particularly well on fine-grained patterns. For both Checker and Fine Stripes, Pixel IoU and Distance Transform produced identical near-perfect solutions, with Pixel IoU converging slightly slower in the case of Fine Stripes. These patterns contain alternating structures at the pixel level, making pixel-wise similarity a natural objective. Because Pixel IoU directly optimizes pixel overlap, it effectively captures these fine structural details.

Distance Transform similarity performed best on patterns emphasizing global structure suggesting that Distance Transform effectively captures large-scale spatial organization. However, it sometimes struggled to maintain crisp boundaries, such as in the Square pattern. Here, it produced a more rounded, blob-like structure, struggling with precise edges.

Unexpectedly, Boundary IoU demonstrated mixed performance. While designed to emphasize structural boundaries, it surprisingly struggled with patterns containing fine details or multiple disconnected components. In the H and Spine patterns, Boundary IoU produced poor visual matches. However, it performed best on the Square pattern, where large, well-defined edges dominate the structure.

Overall, Pixel IoU produced the most consistently accurate visual results, particularly for fine-grained patterns. Distance Transform performed well for global structural alignment but struggled with precise boundaries. Boundary IoU performed best for simple shapes with strong edges but struggled with complex or fine-grained patterns. These findings reinforce that numerical fitness does not always align with perceptual similarity and highlight the importance of selecting fitness functions based on target pattern characteristics.

C. RQ3: CA Runtime Dynamics

Results indicate the existence of an optimal runtime window for cellular automata evolution. Across all fitness functions, optimal performance consistently occurred between 7 and 25 iterations.

This finding suggests that early-stage morphogenetic growth captures stable structural formation, while extended runtimes introduce chaotic behavior that degrades pattern quality.

These findings emphasize the importance of evaluating cellular automata at intermediate stages rather than final states, particularly when evolving dynamic systems.

D. Future Work

Future work may explore improved fitness design, including multi-objective fitness functions that combine pixel-level and structural similarity. Additionally, designing more challenging target patterns may help prevent solutions that exploit weaknesses in individual fitness metrics.

Stability analysis presents an important extension. While this study evaluates optimal CA steps, future work may investigate long-term stability of evolved rules. Additionally, modifying boundary conditions, such as removing periodic wrapping, may further influence pattern formation dynamics.

Cellular automata representation itself may be expanded. Larger neighborhoods or alternative geometric neighborhood structures may allow more complex morphogenetic behavior and improve pattern expressiveness.

Finally, further experimental analysis could investigate population diversity during evolution. Understanding diversity dynamics may provide insight into convergence behavior and improve evolutionary performance.

These directions provide opportunities to improve both evolutionary performance and morphogenetic complexity in future studies.

VI. Conclusion

This study demonstrates that artificial morphogenesis can be achieved using evolutionary cellular automata under appropriate conditions. On small 17×17 grids, evolved CA rule sets were able to generate visually meaningful approximations of target patterns, particularly when evaluated under pixel-based Intersection over Union (IoU) and distance-transform-based fitness functions. Across experiments, we observed that intermediate evaluation depths, typically between 7 and 25 CA iterations, consistently produced the most stable and accurate morphogenetic structures.

However, our results also highlight significant differences between fitness formulations. Pixel IoU and distance transform metrics generally produced stronger visual alignments with target patterns, while boundary-based metrics were less reliable. We also find that high numerical fitness does not always correspond to visually correct outcomes, indicating instances of deceptive convergence.

Overall, these findings reinforce that fitness function design is a critical factor in evolutionary morphogenesis. Even in simple discrete systems, less-sophisticated similarity metrics can lead to structurally incorrect yet highly optimized solutions, underscoring the importance of carefully aligning fitness objectives with intended visual structure.

Code, experiments, and visualizations are publicly available in our project repository.

References

[1] A. Mordvintsev, E. Randazzo, E. Niklasson, and M. Levin, "Growing neural cellular automata," *Distill*, 2020, https://distill.pub/2020/growing-ca.

[2] S. Wolfram, *A New Kind of Science*. Wolfram Media, 2002.

[3] C. G. Langton, "Computation at the edge of chaos," *Physica D*, vol. 42, pp. 12–37, 1990.

[4] M. Mitchell, J. P. Crutchfield, and R. Das, "Evolving cellular automata with genetic algorithms: A review of recent work," in *Handbook of Evolutionary Computation*, T. Bäck, D. Fogel, and Z. Michalewicz, Eds. Oxford, UK: Oxford University Press, 1997.

[5] M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL visual object classes challenge: A retrospective," *International Journal of Computer Vision*, vol. 111, no. 1, pp. 98–136, 2015.

[6] B. Cheng, R. Girshick, P. Dollár, A. C. Berg, and A. Kirillov, "Boundary IoU: Improving object-centric image segmentation evaluation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 15 334–15 342.

[7] G. Borgefors, "Distance transformations in digital images," *Computer Vision, Graphics, and Image Processing*, vol. 34, no. 3, pp. 344–371, 1986.

[8] V. V. Starovoitov, "Morphological analysis based on distance transforms," in *Image Algebra and Morphological Image Processing V*, vol. 2300. SPIE, 1994, pp. 266–277.

[9] P. M. Silva and J. B. Florindo, "Using down-sampling for multiscale analysis of texture images," *Pattern Recognition Letters*, vol. 129, pp. 253–258, 2020.

[10] B. W.-C. Chan, "Towards large-scale simulations of open-ended evolution in continuous cellular automata," 2023. [Online]. Available: https://arxiv.org/abs/2304.05639

[11] H. Sayama, "Self-reproduction and evolution in cellular automata: 25 years after evoloops," *Artificial Life*, vol. 31, no. 1, pp. 81–95, 2024, iEEE Document: 10907982.

[12] Y. Lai, X. Chen, A. Wang, A. Yuille, and Z. Zhou, "From pixel to cancer: Cellular automata in computed tomography," in *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024*, vol. 15001. Springer, Cham, 2024, pp. 36–46.